

NIEBEZPIECZNY PROJEKT A06:2025

definicja:

To kategoria obejmująca różne słabości - brakujący lub nieskuteczny projekt kontroli. Niebezpieczny projekt nie jest źródłem wszystkich pozostałych kategorii ryzyka. Istnieje różnica między bezpiecznym projektem a niebezpieczną implementacją. Występują wady projektowe i wady implementacji. Mają one różne przyczyny źródłowe, pojawiają się na różnych etapach procesu rozwoju i wymagają różnych metod naprawczych. Bezpieczny projekt nadal może zawierać wady implementacji, które prowadzą do luk w zabezpieczeniach, które mogą zostać wykorzystane.

Niebezpiecznego projektu nie da się naprawić za pomocą idealnej implementacji, ponieważ mechanizmy kontroli bezpieczeństwa nigdy nie zostały stworzone w celu obrony przed konkretnymi atakami.

przyczyny występowania:

Jednym z czynników przyczyniających się do niebezpiecznego projektu jest brak profilowania ryzyka biznesowego w opracowanym oprogramowaniu lub systemie, a tym samym brak określenia wymaganego poziomu bezpieczeństwa projektu.

Wady w logice biznesowej aplikacji, np. brak zdefiniowania niepożądanych lub nieoczekiwanych zmian stanu w aplikacji.

Brak współpracy między zespołami - developerzy, security i biznes nie komunikują się efektywnie.

Presja czasu i priorytyzacja funkcjonalności - bezpieczeństwo odkładane na później.

Nieuwzględnienie scenariuszy nadużyć - myślenie tylko o "happy path", ignorowanie tego, jak atakujący może wykorzystać funkcjonalność.

Przykładowe ataki :

Przykład 1: Sklep internetowy - brak limitu prób zakupu towaru

Scenariusz: Sklep wyprzedaje limitowane sneakersy (100 par). System pozwala dodać produkt do koszyka i rezerwuje go na 15 minut podczas finalizacji zamówienia.

Wadliwy projekt:

- Brak limitu liczby równoczesnych koszyków dla jednego użytkownika
- Brak wykrywania zautomatyzowanych botów
- Brak systemu kolejkowania

Atak: Atakujący uruchamia skrypt, który tworzy 100 koszyków jednocześnie, rezerwując wszystkie pary. Prawdziwi klienci widzą "wyprzedane", podczas gdy bot kontroluje całą dostępność.

Poprawny projekt:

- Limit 1 rezerwacja na użytkownika/IP/sesję

- CAPTCHA przy wysokim popycie
- System kolejkowy z weryfikacją
- Analiza zachowania użytkownika

Przykład 2: System resetowania hasła - brak mechanizmu rate limiting

Scenariusz: Aplikacja bankowa pozwala na reset hasła przez email.

Wadliwy projekt:

- Brak limitu żądań resetowania hasła
- Kod resetujący (6 cyfr) nie wygasa przez 24h
- Brak blokady po nieudanych próbach

Atak: Atakujący zna adres email ofiary. Inicjuje reset hasła i uruchamia atak brute-force, testując wszystkie 1,000,000 kombinacji (000000-999999). Bez rate limitingu może to zrobić w kilka godzin.

Poprawny projekt:

- Rate limiting: max 3 żądania resetowania na godzinę
- Kod jednorazowy, wygasający po 15 minutach
- Długi, losowy token zamiast krótkiego kodu
- Blokada konta po 5 nieudanych próbach
- Powiadomienia o podejrzanej aktywności

Przykład 3: System płatności - workflow umożliwiający manipulację ceną

Scenariusz: Platforma e-commerce z procesem płatności wieloetapowym.

Wadliwy projekt:

Krok 1: Użytkownik wybiera produkt (cena: 1000 PLN)

Krok 2: Wybór sposobu dostawy

Krok 3: Formularz płatności - cena przesyłana jako hidden field w HTML

Krok 4: Finalizacja zamówienia

Atak: Atakujący przechwytuje request przed Krokiem 4 i modyfikuje:

html

```
<input type="hidden" name="price" value="1000">
```

```
<!-- zmienia na -->
```

```
<input type="hidden" name="price" value="1">
```

...

System akceptuje zmodyfikowaną cenę, bo walidacja odbywa się tylko po stronie klienta.

****Poprawny projekt:****

- Cena ****nigdy**** nie jest przesyłana z frontendu
- Każde zamówienie ma unikalny ID sesji
- Backend oblicza cenę na podstawie:
 - ID produktu pobranego z zaufanej bazy danych
 - Aktualnych promocji weryfikowanych po stronie serwera
- Integralność danych chroniona tokenami CSRF
- Logowanie wszystkich transakcji z monitoringiem anomalii

Przykład 4: Forum dyskusyjne - brak kontroli dostępu na poziomie logiki biznesowej**

****Scenariusz:****

Forum z sekcjami publicznymi i prywatnymi (np. dla moderatorów).

****Wadliwy projekt:****

- Widoczność wątków kontrolowana tylko przez UI (ukryte linki) - Brak weryfikacji uprawnień przy pobieraniu treści - Przewidywalne ID wątków (1, 2, 3, 4...)

****Atak:**** ``

Normalny użytkownik widzi:

GET /thread/42 ✓ dostępny

Atakujący próbuje: GET /thread/43 ✓ prywatny wątek moderatorski - dostępny!

GET /thread/44 ✓ kolejny wrażliwy wątek

Backend nie sprawdza, czy użytkownik ma prawo dostępu - po prostu zakłada, że skoro ktoś zna URL, to UI go tam puszczało.

Poprawny projekt:

- **Każde** zapytanie do API weryfikuje uprawnienia użytkownika
- ACL (Access Control List) sprawdzane na backendzie
- Losowe, nieprzewidywalne identyfikatory (UUID zamiast sekwencyjnych ID)
- Audit log dostępu do wrażliwych zasobów

- Testy penetracyjne obejmujące scenariusze IDOR (Insecure Direct Object Reference)

Potencjalne ryzyko:

Jakie konsekwencje może nieść dla aplikacji/firmy?

1. Straty finansowe

Bezpośrednie:

- Kradzież pieniędzy (manipulacja cenami, nieuprawnione transakcje)
- Zwroty kosztów dla poszkodowanych klientów
- Kary regulacyjne (RODO, PCI DSS)

Pośrednie:

- Koszt przeprojektowania i przebudowy systemu
- Utracone przychody podczas przestoju
- Wzrost składek ubezpieczeniowych

Przykład: Sklep z wadliwym systemem płatności - atakujący kupują produkty za grosze.
Strata: dziesiątki tysięcy złotych zanim zauważysz problem.

2. Utrata reputacji i zaufania klientów

- Klienci odchodzą do konkurencji
- Negatywne opinie w mediach i social media
- Trudności w pozyskaniu nowych użytkowników
- Spadek wartości akcji (firmy publiczne)

Przykład: Wyciek danych z forum z powodu złego projektu kontroli dostępu - użytkownicy tracą zaufanie, firma zyskuje łatkę "niebezpiecznej".

3. Konsekwencje prawne

- Pozwy sądowe od poszkodowanych użytkowników
- Kary od organów regulacyjnych (UODO, KNF)
- Obowiązek publicznego zgłoszenia incydentu
- Dochodzenia prowadzone przez organy ścigania

Przykład: Bank z wadliwym systemem autoryzacji - nieautoryzowany dostęp do kont klientów = naruszenie RODO + kary finansowe.

4. Szkody operacyjne

- Konieczność wyłączenia usługi do czasu naprawy
- Przeprojektowanie wymaga czasu (tygodnie/miesiące)
- Zasoby techniczne przesunięte z rozwoju na naprawę
- Demotywacja zespołu

Przykład: E-commerce z botem wykupującym wszystkie promocje - musisz zawiesić sprzedaż i stracić sezon świąteczny.

5. Przewaga konkurencji

- Konkurenci wykorzystują Twoje problemy w marketingu
 - Utrata pozycji rynkowej
 - Klienci migrują do bezpieczniejszych alternatyw
-

Podsumowanie: Im poważniejszy błąd w projekcie, tym droższe konsekwencje

 **Kluczowa zasada:** Naprawa błędu projektowego po wdrożeniu kosztuje **100x więcej** niż zaprojektowanie dobrze od początku.

 **Wniosek:** Inwestycja w bezpieczny projekt na starcie = oszczędność milionów później.

Metody zapobiegania :

Jak się przed nią zabezpieczyć?

1. Modelowanie zagrożeń (Threat Modeling)

CO TO JEST: Analiza "co może pójść źle" **PRZED** rozpoczęciem kodowania.

JAK TO ZROBIĆ:

- Narysuj schemat przepływu danych w aplikacji
- Zidentyfikuj punkty krytyczne (login, płatności, dane osobowe)
- Zadaj pytania: "Jak atakujący może to wykorzystać?"
- Użyj metodyki STRIDE lub PASTA

PRZYKŁAD:

Funkcja: Reset hasła przez email

Pytania:

- Co jeśli ktoś wyśle 1000 requestów?
- Co jeśli kod resetujący jest za krótki?
- Co jeśli token nie wygasa?

Rozwiązanie: Rate limiting + długi token + krótki czas życia

2. Zasady bezpiecznego projektowania (Security Design Principles)

PODSTAWOWE ZASADY:

Defense in Depth (obrona warstwowa)

- Nie polegaj na jednym zabezpieczeniu
- Przykład: Login = hasło + 2FA + rate limiting + CAPTCHA

Least Privilege (minimalne uprawnienia)

- Użytkownik dostaje tylko to, czego potrzebuje
- Przykład: Moderator widzi tylko swoje sekcje, nie wszystkie

Fail Secure (bezpieczne awarie)

- Gdy coś się zepsuje, system zamyka dostęp (nie otwiera)
- Przykład: Błąd sprawdzania uprawnień = brak dostępu (nie pełny dostęp)

Zero Trust (nie ufaj nikomu)

- Weryfikuj WSZYSTKO po stronie serwera
- Przykład: Nie ufaj cenom z formularza HTML

3. Kontrola dostępu na poziomie backendu

ZAWSZE weryfikuj uprawnienia po stronie serwera

**** ZASADY:**** - Każde API sprawdza uprawnienia - Nie używaj przewidywalnych ID (UUID zamiast 1,2,3) - Loguj próby nieautoryzowanego dostępu

4. Rate Limiting i zabezpieczenia przed automatyzacją**

**** CO TO CHRONI:****

* Brute force, boty, nadużycia API *

*JAK WDROŻYĆ:*** ``

Login: max 5 prób na 15 minut

Reset hasła: max 3 żądania na godzinę

API: max 100 requestów na minutę na użytkownika

Dodawanie do koszyka: CAPTCHA przy podejrzanej aktywności

5. Użycie sprawdzonych wzorców i frameworków

NIE WYMYŚLAJ KOŁA OD NOWA:

- ✓ Autoryzacja: OAuth 2.0, OpenID Connect
- ✓ Sesje: JWT z podpisem, secure cookies
- ✓ Płatności: Stripe, PayPal (nie buduj własnego!)
- ✓ Rate limiting: Gotowe biblioteki (express-rate-limit, django-ratelimit)

DLACZEGO:

Te rozwiązania są sprawdzone przez miliony użytkowników i regularnie audytowane.

6. Security training dla zespołu

EDUKACJA:

- Szkolenia z OWASP Top 10
- Warsztaty z threat modeling
- Współpraca między dev i security

KULTURA:

- Security to odpowiedzialność każdego, nie tylko security teamu
- "Myśl jak atakujący" podczas projektowania

Checklist przed wdrożeniem:

- Przeprowadzono modelowanie zagrożeń?
- Każda funkcja ma kontrolę dostępu po stronie serwera?
- Wrażliwe operacje mają rate limiting?
- Ceny/kwoty nigdy nie pochodzą z frontendu?
- Przeprowadzono testy abuse cases?
- Code review obejmował aspekty security?
- Istnieją alerty na anomalie?
- Dokumentacja opisuje założenia bezpieczeństwa?

Podsumowanie: Bezpieczeństwo zaczyna się od projektu

💡 **Najważniejsza zasada:** Łatwiej zaprojektować bezpiecznie od początku, niż naprawiać później.

🎯 **Kluczowa zmiana myślenia:** Z "jak to ma działać?" na "jak ktoś może to złamać?"

Przykład z życia :

Incident: Ticketmaster (2024) - Wadliwy system kolejkowania

Co się stało?

W maju 2024 roku grupa hakerów ShinyHunters ukradła dane 560 milionów klientów Ticketmaster z bazy danych Snowflake. Jednak prawdziwy problem projektowy ujawnił się wcześniej.

Problem projektowy:

System kolejkowania biletów Ticketmaster miał fundamentalne wady:

- 1. Brak ochrony przed botami na etapie projektowania**
 - System rezerwował bilety na podstawie "kto pierwszy, ten lepszy"
 - Boty mogły tworzyć tysiące wirtualnych sesji jednocześnie
 - Brak mechanizmu wykrywania zautomatyzowanego ruchu
- 2. Przewidywalne tokeny sesji**
 - Tokeny kolejkowe były generowane w przewidywalny sposób
 - Atakujący mogli "przeskoczyć" kolejkę manipulując tokenami
- 3. Brak rate limitingu podczas największego ruchu**
 - Podczas sprzedaży biletów na koncerty Taylor Swift (2023-2024) system padł
 - Miliony fanów nie mogło kupić biletów, które wykupili botownicy

Konsekwencje:

💰 Finansowe:

- Kary regulacyjne od Departamentu Sprawiedliwości USA
- Pozwy zbiorowe od milionów klientów
- Strata wartości akcji Live Nation (właściciel Ticketmaster)

⚖️ Prawne:

- Dochodzenie Kongresu USA w 2023 r.
- Postępowanie antymonopolowe
- Kary za naruszenie praw konsumentów

🤨 Reputacyjne:

- Masowe protesty fanów
- Trending #TicketmasterScalpers przez tygodnie
- Taylor Swift publicznie skrytykowała firmę
- Utrata zaufania całej branży muzycznej

🏛️ Regulacyjne:

- Propozycje nowych przepisów "BOSS Act" w USA
- Zakaz odsprzedaży przez boty na poziomie federalnym

Co było źle zaprojektowane?

WADLIWY PROJEKT:

1. Brak weryfikacji "jesteś człowiekiem" przy dużym ruchu
2. Prosty system kolejkowy bez ochrony anty-botowej
3. Przewidywalne identyfikatory sesji
4. Brak limitu biletów na użytkownika/IP/kartę
5. Możliwość masowego rezerwowania przez API

POPRAWNY PROJEKT powinien mieć:

- ✓ CAPTCHA/Cloudflare Turnstile obowiązkowe
- ✓ Weryfikacja numeru telefonu przed dostępem
- ✓ Losowe, kryptograficzne tokeny kolejkowe
- ✓ Limit 4-6 biletów na klienta (weryfikowany po karcie + adresie)
- ✓ AI wykrywające wzorce botów w czasie rzeczywistym
- ✓ System "sprawiedliwej kolejki" losujący dostęp zamiast "wyścigu"

Dlaczego to jest "Insecure Design"?

- ✗ Problem nie był w **kodzie** (implementacji)
- ✓ Problem był w **architekturze** systemu - nawet perfekcyjny kod nie uratowałby tego projektu

Ticketmaster wiedział o problemie od lat, ale zamiast przeprojektować system, stosowali "patche":

- Dodawali CAPTCHA post-factum (nieskuteczne)
- Blokowali pojedyncze IP (botownicy używali proxy)
- Zwiększali serwery (problem nie w wydajności, a w logice!)

Lekcja:

"Nie można naprawić złego projektu łatkami."

System sprzedaży biletów wymagał **fundamentalnego przeprojektowania** z uwzględnieniem:

- Modelowania zagrożeń (threat modeling) - "jak botownicy mogą nadużyć systemu?"
- Zasady defense in depth - wielowarstwowa ochrona
- Testów abuse cases - symulacje ataków botów przed wdrożeniem

Aktualizacja 2025:

Po skandalach Ticketmaster **częściowo przeprojektował** system:

- Wprowadzono Verified Fan (weryfikacja tożsamości)
- Dynamiczne ceny utrudniające botom
- Lepsza integracja z Cloudflare Bot Management

Ale **szkody już powstały** - koszty naprawy to dziesiątki milionów dolarów i zniszczona reputacja.

Wniosek:

Koszt złego projektu:

- Zaprojektowanie dobrze od początku: ~500k USD
- Naprawa po skandalu: ~50-100 mln USD + utracona reputacja

100x drożej naprawić niż zrobić dobrze od razu! 🎯